
QUAID-E-AWAM UNIVERSITY OF ENGINEERING SCIENCE & TECHNOLOGY

Python Programming Bootcamp - Week 4, Day 01

Mastering Object-Oriented Programming in Python

Atta Muhammad

attapanhyar@quest.edu.pk

+923331971110

Quaid-e-Awam University of Engineering, Science and Technology

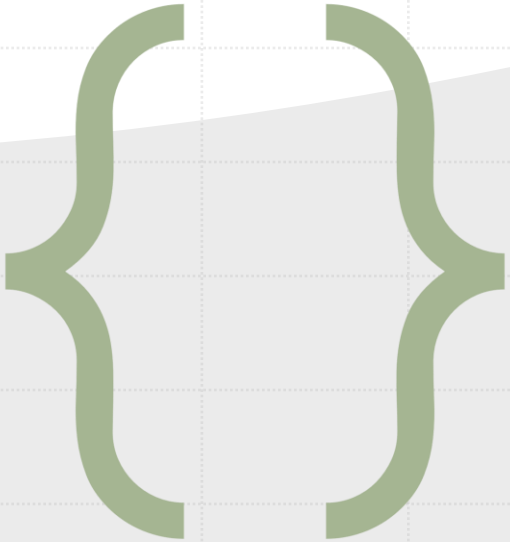
Today's objective

- OOP Basics
- Classes and Objects
- Inheritance
- Encapsulation
- Polymorphism
- Hands-on Exercises

What is Object-Oriented Programming?

- Object-oriented programming (OOP) is a programming paradigm based on the concept of objects which can contain data and code
- Benefits of using OOP
- Key principles: **Abstraction**, **Encapsulation**, **Inheritance**, **Polymorphism**.

Why Use OOP in Python?



- **Modular** and **reusable** code.
- Easier to manage complexity.
- Widely used in real-world applications like **GUI development**, game development, etc.

Classes and Objects

- **Class**: A blueprint for creating objects.
- **Object**: An instance of a class.
- Syntax for defining a class and creating an object.

Classes and Objects

- **Syntax for defining a class and creating an object.**

```python
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def make_sound(self):
        print(f'{self.name} says {self.sound}')

# Create an object
dog = Animal('Dog', 'Woof')
dog.make_sound()  # Output: Dog says Woof
```

Attributes and Methods

- **Attributes**: Variables that belong to the object or class.
- **Methods**: Functions that belong to the class.
- **Instance Attributes** vs. **Class Attributes** with examples.

Instance Attributes vs. Class Attributes

```python
class Car:
    # Class attribute (shared across all instances)
    wheels = 4
    # Instance attributes (specific to each instance)
    def __init__(self, make, model, year):
        self.make = make  # instance attribute
        self.model = model  # instance attribute
        self.year = year  # instance attribute
    # Instance method
    def start_engine(self):
        print(f'The engine of {self.make} {self.model} ({self.year}) has started.')
    # Class method (working with class attributes)
    @classmethod
    def car_info(cls):
        print(f"Cars usually have {cls.wheels} wheels.")
car1 = Car('Toyota', 'Corolla', 2020)
car2 = Car('Honda', 'Civic', 2019)
# Accessing instance attributes and methods
car1.start_engine()  # Output: The engine of Toyota Corolla (2020) has started.
car2.start_engine()  # Output: The engine of Honda Civic (2019) has started.
# Accessing class method and class attribute
Car.car_info()  # Output: Cars usually have 4 wheels.
```

Hands-on Exercise: Creating Your First Class

- Create a **Car** class with attributes like **make, model**, and **year**.

- Add a method **start_engine** that prints "**Engine started**."

Object Representation

- **__repr__**(self): Should return an unambiguous string representation of the object, ideally one that could be used to recreate the object.

- **__str__(self):** Returns a user-friendly string representation of the object.

Object Representation

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, aged {self.age}"

    def __repr__(self):
        return f"Person('{self.name}', {self.age})"

p = Person("Alie", 30)
print(str(p))  # Output: Ali, aged 30
print(repr(p)) # Output: Person('Ali', 30)
```

Comparison Magic Methods

- **__eq__(**self, other): Equality (==).
- **__ne__(**self, other): Inequality (!=).
- **__lt__(**self, other): Less than (<).
- **__le__(**self, other): Less than or equal to (<=).
- **__gt__(**self, other): Greater than (>).
- **__ge__(**self, other): Greater than or equal to (>=)

Comparison Magic Methods

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __eq__(self, other):
        return self.title == other.title and self.author == other.author

book1 = Book("1984", "George Orwell")
book2 = Book("1984", "George Orwell")
print(book1 == book2)  # Output: True
```

Inheritance

- What is inheritance?

- Benefits of code reusability.

Inheritance – Example

```
Animal:
f __init__(self, name):
    self.name = name

ef make_sound(self):
    pass

ss Dog(Animal):
  def make_sound(self):
      print(f'{self.name} says Woof')

class Cat(Animal):
    def make_sound(self):
        print(f'{self.name} says Meow')

dog = Dog('Buddy')
cat = Cat('Kitty')
dog.make_sound()  # Buddy says Woof
:at.make_sound()  # Kitty says Meow
```

Hands-on Exercise: Inheritance

- Extend the **Car** class to create a **ElectricCar** class that adds battery-specific attributes

Encapsulation

- Protecting data by limiting access to methods.

- Use of private (_ or __) attributes.

Encapsulation

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance  # private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

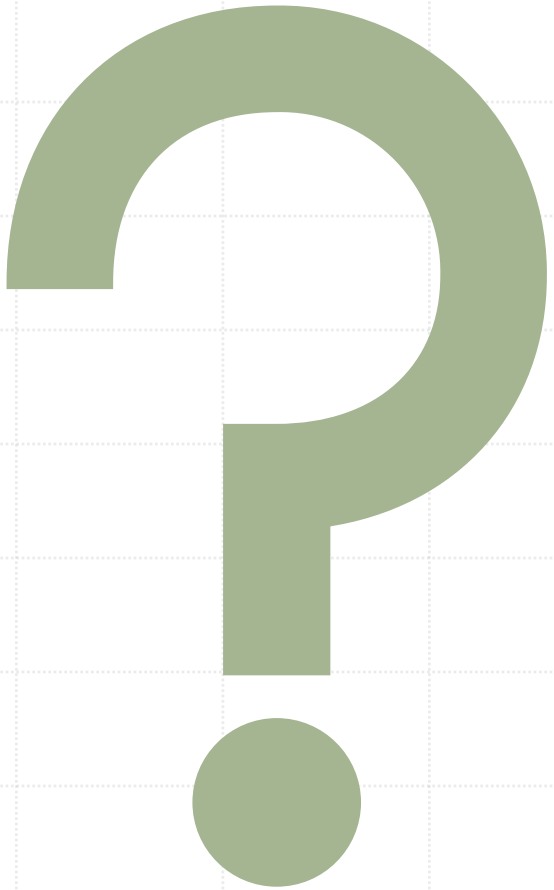
Polymorphism

- Ability to use a common interface for multiple types.

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2
```

Hands-on Exercise: Polymorphism

- Create a parent class **Shape** and derived classes **Rectangle** and **Triangle** with their own **area()** method.



Questions ?